

Guía de Estudio: Patrón MVC en CodeIgniter 4

Aprende paso a paso cómo viajan los datos desde la Base de Datos hasta la Vista para rendir y aprobar tu examen.

1. ¿Qué es el Patrón MVC y cuál es el rol de cada parte?

El patrón **MVC (Modelo - Vista - Controlador)** es una arquitectura de software que separa la aplicación en tres capas independientes con responsabilidades únicas:

Analogía del Restaurante:

- **El Cliente (Navegador / Usuario):** Pide un plato ingresando a una URL (por ejemplo: `/dashboard` o `/plantas`).
- **El Controlador (El Mozo / Mesero):** Recibe la petición del usuario, va a la cocina, pide lo que necesita, prepara la bandeja con los datos (`$data`) y se la lleva al cliente. **Coordina todo.**
- **El Modelo (El Chef / Despensa):** Es el **único** que interactúa con los ingredientes (**la Base de Datos MySQL**). Sabe hacer consultas SQL (`SELECT` , `INSERT` , `UPDATE` , `DELETE`).
- **La Vista (El Plato servido y decorado):** Es el código **HTML / CSS** que muestra visualmente los datos listos en la pantalla del usuario.

Diagrama del Flujo de Datos (El Viaje del Dato)

```
[ USUARIO ]
  | (1. Ingres a la URL, ej: /dashboard o /plantas)
  ▼
[ RUTA (app/Config/Routes.php) ]
  |
  ▼
[ CONTROLADOR (app/Controllers/SmartPlant.php) ]
  | (2. Pide datos: $plantaModel->activasPorUsuario())
  ▼
[ MODELO (app/Models/PlantaModel.php) ] → [ BASE DE DATOS (MySQL) ]
  |                                     | (3. Ejecuta SELECT seguro)
  |←───────────────────────────────────| (4. Devuelve array con registros)
(5. Inyecta el array $data)
  |
  ▼
[ VISTA (app/Views/Dashboard.php) ] → [ PANTALLA DEL USUARIO ] (6. Renderiza HTML)
```

2. Paso a Paso: El Código Real de tu Proyecto y Qué Hace Cada Línea

PASO 1: EL MODELO (M)

Archivo: `app/Models/PlantaModel.php`

Representa a la tabla `plantas` en MySQL. Es el único archivo que ejecuta consultas a la base de datos.

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class PlantaModel extends Model
{
    // 1. Nombre exacto de la tabla en MySQL
    protected $table = 'plantas';

    // 2. Clave primaria única
    protected $primaryKey = 'id';

    // 3. Formato en que devuelve los registros ('array' o 'object')
    protected $returnType = 'array';

    // 4. Columnas habilitadas para consultas y operaciones
    protected $allowedFields = [
        'usuario_id', 'nombre', 'especie', 'descripcion',
        'humedad_min', 'humedad_max', 'temp_min', 'temp_max', 'activa'
    ];

    /**
     * Consulta las plantas activas que pertenecen a un usuario específico
     */
    public function activasPorUsuario(int $usuarioId): array
    {
        // SQL equivalente: SELECT * FROM plantas WHERE usuario_id = $usuarioId AND activa = 1 ORDER BY id ASC;
        return $this->where('usuario_id', $usuarioId)
            ->where('activa', 1)
            ->orderBy('id', 'ASC')
            ->findAll();
    }
}
```

Explicación de cada línea del Modelo:

- `class PlantaModel extends Model` : Hereda todas las funciones de base de datos nativas de CodeIgniter 4 (ORM / Query Builder).
- `protected $table = 'plantas';` : Le dice al framework con qué tabla de MySQL debe operar.
- `->where(...)->findAll()` : Ejecuta la consulta SQL con parámetros preparados (Prepared Statements) que evitan inyecciones SQL.

PASO 2: LA RUTA (ROUTES)**Archivo:** `app/Config/Routes.php`

Conecta la URL del navegador con el Controlador que debe responder.

```
// Cuando el usuario entra a http://localhost:8080/dashboard
$routes->get('dashboard', 'SmartPlant::dashboard');
$routes->get('plantas', 'Plantas::index');
```

**Explicación de la línea de Ruta:**

- `$routes->get(...)` : Escucha peticiones HTTP de tipo GET.
- `'dashboard'` : Es el camino o URL que escribe el usuario.
- `'SmartPlant::dashboard'` : Indica la clase controladora (`SmartPlant`) y el método (`dashboard`) a ejecutar.

PASO 3: EL CONTROLADOR (C)**Archivo:** `app/Controllers/SmartPlant.php` (o `Plantas.php`)

El intermediario. Recibe la petición, llama al Modelo para obtener los datos, los empaqueta en `$data` y llama a la Vista.

```
<?php

namespace App\Controllers;

// 1. Importamos la clase del Modelo
use App\Models\PlantaModel;

class SmartPlant extends BaseController
{
    public function dashboard()
    {
        // 2. Obtenemos el ID del usuario en sesión
        $usuario_id = (int) ($_SESSION['usuario_id'] ?? 1);

        // 3. Instanciamos el Modelo
        $plantaModel = new PlantaModel();

        // 4. Le pedimos los datos a la base de datos a través del Modelo
        $misPlantas = $plantaModel->activasPorUsuario($usuario_id);

        // 5. Empaquetamos los datos en el array asociativo $data
        $data = [
            'titulo' => 'Panel Principal - SmartPlant CARE',
            'plantas' => $misPlantas // Esta clave 'plantas' se convertirá en $plantas en la vista
        ];

        // 6. Cargamos la vista y le inyectamos los datos
        return view('Dashboard', $data);
    }
}
```

 Explicación de cada línea del Controlador:

- `$plantaModel = new PlantaModel();` : Crea el objeto para solicitar información al modelo.
- `$data = ['plantas' => $misPlantas];` : **Clave para el examen:** Toda variable que deba mostrarse en pantalla debe viajar dentro de este arreglo asociativo.
- `return view('Dashboard', $data);` : Renderiza la vista y le transfiere los datos limpios.

PASO 4: LA VISTA (V)**Archivo:** `app/Views/Dashboard.php`

Presentación visual. Recibe las variables inyectadas y las dibuja con HTML y estructuras PHP simples (`foreach` , `if`).

```

<?php
// Recibe las variables provistas por el Controlador
$plantas = $plantas ?? [];
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title><?= esc($titulo) ?></title>
</head>
<body>
    <h1><?= esc($titulo) ?></h1>

    <!-- Verificamos si hay registros -->
    <?php if (!empty($plantas) && is_array($plantas)): ?>
        <table border="1" cellpadding="8">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Nombre</th>
                    <th>Especie</th>
                    <th>Humedad Mín/Máx</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($plantas as $planta): ?>
                    <tr>
                        <td><?= esc($planta['id']) ?></td>
                        <td><strong><?= esc($planta['nombre']) ?></strong></td>
                        <td><?= esc($planta['especie']) ?></td>
                        <td><?= esc($planta['humedad_min']) ?>% - <?= esc($planta['humedad_max']) ?>%</td>
                    </tr>
                <?php endforeach; ?>
            </tbody>
        </table>
    <?php else: ?>
        <p>No se encontraron plantas registradas en la base de datos.</p>
    <?php endif; ?>
</body>
</html>

```



Regla de Oro para el Examen: En la Vista **NUNCA** debe haber llamadas a `Database::connect()` , sentencias `SELECT` , ni `$stmt->prepare()` . La vista solo muestra los datos que el Controlador le entregó.

3. El "Molde Universal" de tu Proyecto (Aplica a todas las tablas)

Todas las secciones de tu página web siguen exactamente esta misma estructura de 4 pasos:

Módulo	1. Modelo (BD)	2. Ruta (URL)	3. Controlador (Lógica)	4. Vista (HTML)
Plantas	PlantaModel.php	/dashboard	\$plantaModel->activasPorUsuario()	Dashboard.php (Recibe \$plantas)
Usuarios / Perfil	UsuarioModel.php	/login	\$usuarioModel->verificarLogin()	Login.php / Dashboard.php
Dispositivos / Hardware	DispositivoModel.php	/hardware/connect	\$dispositivoModel->obtenerPorUsuario()	HardwareConnect.php (Recibe \$devices)
Sensores / Telemetría	LecturaModel.php	/dashboard	\$lecturaModel->obtenerUltimaLectura()	Dashboard.php (Recibe \$humedad , \$temp)
Alertas / Eventos	EventoModel.php	/dashboard	\$eventoModel->obtenerRecientes()	Dashboard.php (Recibe \$eventos)
Soporte	UsuarioModel.php	/support	Valida correo y mensaje	Support.php (Recibe \$msg_estado)

4. Simulador de Preguntas y Respuestas para el Examen Oral 🎓

Pregunta del Profesor	Respuesta Correcta
¿Por qué está mal colocar una consulta <code>SELECT</code> dentro del archivo de una Vista?	Porque violaría el principio de <i>separación de responsabilidades</i> . La Vista solo debe encargarse de la presentación visual. Si colocamos consultas SQL en la vista, exponemos la seguridad de la base de datos y el código se vuelve imposible de mantener y reutilizar.
¿Quién conecta la Vista con el Modelo?	El Controlador . El Modelo y la Vista nunca deben comunicarse directamente entre sí. El Controlador recibe la petición, le solicita los datos al Modelo y se los entrega a la Vista empaquetados en un array asociativo <code>\$data</code> .
¿Cómo viajan los datos desde el Controlador a la Vista en CodeIgniter 4?	A través del arreglo asociativo <code>\$data</code> que se le pasa a la función <code>view('nombre_vista', \$data)</code> . Cada clave del arreglo se convierte automáticamente en una variable con ese mismo nombre dentro del archivo de la vista.
¿Qué ventajas tiene usar <code>CodeIgniter\Model</code> frente a consultas SQL manuales?	Proporciona abstracción de datos (ORM y Query Builder), mapeo seguro de columnas (<code>allowedFields</code>) y protección automática contra inyecciones SQL gracias al uso de sentencias preparadas.